

Repeated String Match

686. Repeated String Match

Let's build the intuition from scratch rather than jumping directly to the trick.

1. What is the problem asking?

You have two strings:

- **a** → the string we are allowed to repeat
- **b** → the string we want to find

We need to find the **minimum number of times a must be repeated** so that **b** becomes a **substring** of the resulting string.

Example

```
a = "abcd"
b = "cdabcdab"
```

Repeat a:

```
1 time → "abcd"
2 times → "abcdabcd"
3 times → "abcdabcdabcd"
```

Now:

"abcdabcdabcd"
"cdabcdab"

So answer = **3**.

2. Key observation

The important question is:

How many copies of a do we need before the generated string is at least as long as b ?

Suppose:

```
a = "abcd"      length = 4
b = "cdabcdab"  length = 8
```

We need at least:

$$\text{ceil}(8 / 4) = 2$$

copies.

But 2 copies give:

"abcdabcd"

Does it contain **b**?

"abcdabcd"
cdabcdab ❌

No.

So we need **one extra repetition**:

```
"abcdabcdabcd"
```

Now it works.

Therefore, the answer can only be:

```
ceil(|b| / |a|)
```

or

```
ceil(|b| / |a|) + 1
```

Why only one extra?

Because **b** may start near the end of one copy of **a** and continue into the next copy.

3. Visual intuition

Think of **a** as an infinitely repeating pattern:

```
a = "abcd"
∞ repeated:
abcd | abcd | abcd | abcd | abcd
  ↑
```

We want to place **b** somewhere inside this infinite pattern.

For:

```
b = "cdabcdab"
```

we can see:

```
abcd | abcd | abcd
  └──────────┘
    c d a b c d a b
```

The substring starts at:

```
c
```

which is near the end of the first "**abcd**".

That's why we need the **next copy** to complete it.

4. Brute-force idea

The simplest approach is:

```
repeat a
check whether b is substring
repeat a
check again
repeat a
check again
...
```

For example:

```
a = "abcd"
b = "cdabcdab"
```

Step 1

```
s = "abcd"
```

Check:

```
"cdabcdab" ∈ "abcd" ?
```

No.

Step 2

```
s = "abcdabcd"
```

Check again.

No.

Step 3

```
s = "abcdabcdabcd"
```

Now yes.

Answer:

```
3
```

This is easy, but we can make the reasoning tighter.

5. Important mathematical observation

Let:

```
n = a.length()
m = b.length()
```

We initially need:

```
ceil(m / n)
```

copies.

In C++:

```
C++
int repeats = (m + n - 1) / n;
```

For example:

```
m = 8
n = 4
(8 + 4 - 1) / 4
= 11 / 4
= 2
```

So initially:

```
repeats = 2
```

But we must check:

```
a repeated 2 times
```

If **b** isn't there, add **one more copy**.

```
repeats++;
```

Then check again.

If still not found:

```
return -1;
```

6. Why do we need at most ONE extra copy?

This is the most important part of the intuition.

Suppose:

```
a = "abc"
```

Infinite repetition:

```
abcabcabcabc...
```

Suppose **b** has length **m**.

Once we've generated enough copies so that the generated string has length at least **m**, any possible occurrence of **b** can start at some position inside one copy of **a**.

The worst case is:

```
start near the END of one copy
```

Then **b** may need the **next copy** of **a**.

Example:

```
a = "abcd"
abcd | abcd | abcd
  ↑
  c d a b c d a b
```

We need the next copy, but we don't need two additional copies.

So:

```
minimum required copies
      ↓
ceil(|b| / |a|)
      ↓
check
      ↓
if not found
      ↓
add ONE copy
      ↓
check again
      ↓
if not found → impossible
```

7. Optimal approach

The algorithm becomes:

Step 1

Calculate the minimum repetitions needed based on length:

```
C++
repeats = ceil(b.length() / a.length());
```

Step 2

Construct **a** repeated **repeats** times.

Step 3

Check:

```
C++
s.find(b)
```

If found:

```
return repeats
```

Step 4

Add one more **a**.

```
C++
s += a;
repeats++;
```

Check again.

If found:

```
return repeats
```

Step 5

Otherwise:

```
return -1
```

8. C++ Code

```
C++
class Solution {
public:
    int repeatedStringMatch(string a, string b) {
        int n = a.size();
        int m = b.size();
```

```
// Minimum repetitions needed based on length
int repeats = (m + n - 1) / n;

string s = "";

// Build the string
for (int i = 0; i < repeats; i++) {
    s += a;
}

// Check with minimum repetitions
if (s.find(b) != string::npos) {
    return repeats;
}

// One extra repetition may complete the substring
s += a;
repeats++;

if (s.find(b) != string::npos) {
    return repeats;
}

// Impossible
return -1;
};
```

9. How the code works

Let's understand each important part.

Calculate repetitions

```
C++
int repeats = (m + n - 1) / n;
```

This calculates:

```
ceil(m / n)
```

without using floating point.

For example:

```
m = 10
n = 4

(10 + 4 - 1) / 4
= 13 / 4
= 3
```

We need at least 3 copies.

Build the repeated string

```
C++
for (int i = 0; i < repeats; i++) {
    s += a;
}
```

If:

```
a = "abcd"
repeats = 2
```

we get:

```
s = "abcdabcd"
```

Check substring

```
C++
if (s.find(b) != string::npos)
```

`find()` searches for `b` inside `s`.

If found:

```
C++
s.find(b)
```

returns its starting index.

If not found:

```
C++
string::npos
```

is returned.

10. Complete dry run

Consider:

```
a = "abcd"
b = "cdabcdab"
```

Initial values

```
a = "abcd"
b = "cdabcdab"

n = 4
m = 8
```

Calculate:

```
repeats = (m + n - 1) / n
         = (8 + 4 - 1) / 4
         = 11 / 4
         = 2
```

So initially we need:

```
2 repetitions
```

Build s

First iteration:

```
s = "abcd"
```

Second iteration:

```
s = "abcdabcd"
```

So:

```
s = "abcdabcd"
```

Now search for:

```
b = "cdabcdab"
```

Inside:

```
abcdabcd
```

Try to visualize:

```
a b c d a b c d
  c d a b c d a b
```

We don't have enough characters after the starting c.

Therefore:

```
NOT FOUND
```

Add one more repetition

```
C++
s += a;
```

Now:

```
s = "abcdabcdabcd"
```

And:

```
repeats = 3
```

Search:

```
b = "cdabcdab"
```

Now:

```
a b c d a b c d a b c d
   |-----|
   c d a b c d a b
```

Found!

Therefore:

```
answer = 3
```

11. Example 2

```
a = "a"
b = "aa"
```

Lengths:

```
n = 1
m = 2
```

Minimum repetitions:

```
ceil(2 / 1) = 2
```

Build:

```
s = "aa"
```

b = "aa" is already a substring.

Therefore:

```
answer = 2
```

12. What if it is impossible?

Consider:

```
a = "abc"
b = "def"
```

Repeat:

```
abcabc
```

No **d**, **e**, or **f** exists.

Add another:

```
abcabcabc
```

Still impossible.

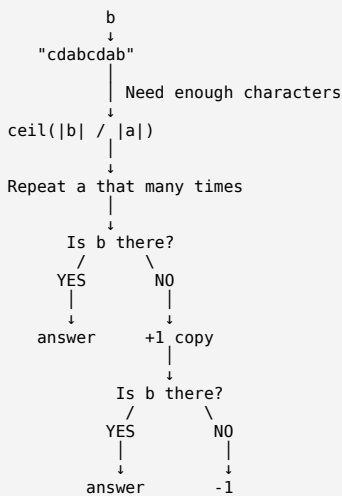
So:

```
return -1
```

The key reason is that repeating a can **never create a character that doesn't exist in a**.

13. Mental model

Remember the problem like this:



The entire trick is:

First get enough length, then allow one extra repetition for boundary crossing.

14. Complexity

Let:

```

n = |a|
m = |b|
  
```

The constructed string has length $O(m + n)$ because we need approximately m/n copies and possibly one extra copy.

`string::find()` is commonly analyzed as $O(m \cdot (m+n))$ in the worst case for the standard library's general substring search behavior, though implementations can be faster in practice.

So for these constraints (`a.length, b.length <= 10^4`), this approach is easily fast enough.

Space

The repeated string uses:

```
O(m + n)
```

space.

★ Final intuition to remember

Don't think:

"How do I compare every possible repetition?"

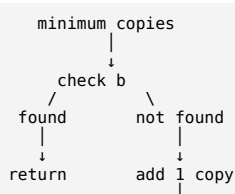
Think:

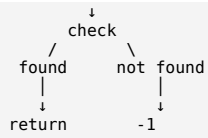
"How many copies are required just to have enough characters to contain b?"

That gives:

```
ceil(|b| / |a|)
```

Then:





This "**minimum required + one extra for boundary crossing**" pattern is the core idea behind LeetCode 686.

ChatGPT can make mistakes. Check important info.